Practical Examination Question Paper

MongoDB, Express.js, Node.js & EJS

Subject: MERN Backend Development
Maximum Marks: 60

General Instructions

1. Use Node.js and Express.js to build the application.
2. Use MongoDB with Mongoose for database operations.
3. Follow the MVC architecture wherever specified.
4. Write clean, modular code with proper routing and error handling.
5. For Question 1, do not create any UI or view files.

Question 1: MVC Without UI (20 Marks)

Domain: Student Management System

Create a Student Management System using Node.js, Express.js, MongoDB, and Mongoose by following MVC architecture without a view.

Requirements

1. Create a MongoDB schema for students with the following fields:
    - name – String, required
    - email – String, required and unique
    - age – Number
    - course – String
    - fees – Number
2. Create the following files:
    - models/Student.js
    - controllers/studentController.js
    - routes/studentRoutes.js
    - db.js

- app.js

3. Implement the following API endpoints:
    - POST /students – Add a new student.
    - GET /students – Display all students as JSON.
    - GET /students/:id – Display a student by ID.
    - PUT /students/:id – Update student details.
    - DELETE /students/:id – Delete a student.
4. Connect the application to MongoDB using db.js.
5. Use appropriate HTTP status codes and handle invalid IDs and database errors.

Note: No EJS, HTML, CSS, or frontend UI is required. Test the APIs using Postman or a similar tool.

Question 2: EJS Templates (15 Marks)

Domain: Product Display System

Given the following array of product objects, display the products using EJS templates.

```
const products = [
 {
  id: 1,
  name: "Laptop",
  category: "Electronics",
  price: 55000
 },
 {
  id: 2,
  name: "Mobile",
  category: "Electronics",
```

```
    price: 20000
  },
  {
    id: 3,
    name: "Headphones",
    category: "Accessories",
    price: 1500
  },
  {
    id: 4,
    name: "Keyboard",
    category: "Accessories",
    price: 2500
  }
];
```

Requirements

1. Create an Express application and configure EJS as the view engine. (3 marks)
2. Pass the products array from the route or controller to the EJS template. (2 marks)
3. Display all products in an **ordered list** using EJS looping syntax. (3 marks)
4. Display all products in an HTML **table** with columns for ID, name, category, and price. (3 marks)
5. Display all products using **Bootstrap cards**, including the product name, category, and price. (4 marks)

Note: Use EJS tags such as <%= %> and <% %> appropriately. Use Bootstrap classes for styling the cards.

Question 3: MVC With UI – Product Management System (25 Marks)

Domain: Product Inventory Management

Develop a complete CRUD application using Node.js, Express.js, MongoDB, Mongoose, and EJS.

Requirements

A. Model and Database (5 marks)

1. Create a Product schema with the following fields:
    - name – String, required
    - description – String
    - price – Number, required
    - category – String
    - quantity – Number
2. Connect the application to MongoDB through db.js.

B. Controller and Router (5 marks)

1. Create a product controller with functions for displaying, adding, updating, and deleting products.
2. Create a product router and connect it to app.js.

C. User Interface Using EJS (5 marks)

Create the following EJS pages:

- products.ejs – Display all products in a table.
- addProduct.ejs – Form to add a product.
- editProduct.ejs – Form to update an existing product.

Use Bootstrap to style the interface.

D. CRUD Operations (10 marks)

Implement the following functionality:

1. **Display Products:** Show all products in a table with Edit and Delete buttons. (2.5 marks)
2. **Add Products:** Create a form to enter product details and save them to MongoDB. (2.5 marks)

3. **Update Products:** Load the selected product into an edit form and save the updated details. (2.5 marks)
4. **Delete Products:** Delete the selected product from MongoDB and redirect to the product list. (2.5 marks)

Suggested Routes

Method	Route	Purpose
GET	/products	Display all products
GET	/products/add	Display the add form
POST	/products/add	Save a new product
GET	/products/edit/:id	Display the edit form
POST	/products/edit/:id	Update product details
POST	/products/delete/:id	Delete a product

Expected Project Structure

```
project/
|
├── models/
|   └── Product.js
|
├── controllers/
|   └── productController.js
|
├── routes/
|   └── productRoutes.js
|
```

├── views/

| ├── products.ejs

| ├── addProduct.ejs

| └── editProduct.ejs

|

├── db.js

├── app.js

├── package.json

└── .env

Evaluation Criteria

Criteria	**Marks**
Question 1 – MVC without UI	20
Question 2 – EJS templates	15
Question 3 – MVC with UI and CRUD	25
Total	**60**